INSTITUTO FEDERAL DE SÃO PAULO
Câmpus Hortolândia

LABORATÓRIO

Montador Daedalus

Programação em linguagem de montagem nos processadores Ahmes e Ramses

AUTOR

Gabriel Cabral Wallendzu
HT3080251

DISCIPLINA

Sistemas de Computação 1 de Automação

PROFESSOR

Carlos Eduardo Pagani

Hortolândia – SP | 2026

Sumário

Resumo 3
1 Introdução 3
2 Objetivos 3
3 Metodologia 4
4 Desenvolvimento 4
Exercício 1 4
Exercício 2 5
Exercício 3 7
Exercício 4 9
Exercício 5 10
Exercício 6 12
Exercício 7 13
5 Testes realizados 14
6 Divisão das tarefas 15
7 Conclusão 16
8 Repositório e referências 16

Resumo

Este relatório documenta a resolução do laboratório “Montador Daedalus”, no qual sete exercícios de programação em linguagem de montagem foram desenvolvidos para os processadores didáticos Ahmes e Ramses. Os programas foram escritos e montados no Daedalus e executados nos respectivos simuladores, de modo a observar o comportamento de registradores, flags e memória. Para cada exercício são apresentados o enunciado, o código comentado nas duas arquiteturas, a explicação da lógica adotada e o resultado esperado, destacando as diferenças entre as duas máquinas.

1 Introdução

O estudo de arquitetura e organização de computadores ganha clareza quando o conceito é colocado em prática em máquinas simples. Ahmes e Ramses são processadores de 8 bits criados para fins didáticos: pequenos o bastante para que cada instrução seja compreendida, mas completos o suficiente para exigir raciocínio sobre registradores, flags, modos de endereçamento e organização da memória. O Daedalus é o montador que traduz o código-fonte em linguagem de montagem para o código de máquina que os simuladores executam.

As duas máquinas têm 256 posições de memória e operam com números de 8 bits, mas diferem em pontos que influenciam diretamente a forma de resolver cada problema. A tabela a seguir resume as diferenças que mais aparecem neste trabalho.

Característica	Ahmes	Ramses
Registradores	Apenas o acumulador (AC)	RA, RB e RX
Flags principais	N, Z, C (além de V e B)	N, Z e C
Modos de endereçamento	Direto	Direto, indireto, imediato e indexado
Chamada de sub-rotina	Não possui JSR	JSR, com retorno por salto indireto
Saltos condicionais	JN, JZ, JNZ, JC, JNC, entre outros	JN, JZ e JC

Como o Ahmes só tem acumulador e endereçamento direto, algumas tarefas que no Ramses são naturais (percorrer uma área de memória, por exemplo) exigem soluções diferentes, como a modificação do operando de uma instrução em tempo de execução. Esses casos são explicados nos exercícios correspondentes.

2 Objetivos

O objetivo geral é resolver os sete exercícios do laboratório “Montador Daedalus” utilizando os simuladores Ahmes e Ramses. Como objetivos específicos, busca-se:

- praticar a escrita de programas em linguagem de montagem e a sintaxe do montador Daedalus;
- compreender o papel das flags (N, Z e C) em comparações, laços e tratamento de overflow;

- comparar o conjunto de instruções e os modos de endereçamento das duas arquiteturas;
- aplicar representações numéricas (complemento de dois, sinal-magnitude e ASCII) diretamente na memória.

3 Metodologia

Cada programa foi escrito no Daedalus, selecionando-se a máquina alvo (Ahmes ou Ramses), montado e executado no simulador correspondente, conferindo ao final o conteúdo da memória e dos registradores. Foi adotada a mesma convenção em todos os programas: as variáveis são declaradas a partir da posição 128 (80h), com a diretiva org 128 e a pseudo-instrução DB, e o código começa na posição 0, com org 0. Todos os programas possuem comentários após o ponto e vírgula, explicando cada etapa, e os códigos-fonte completos estão no repositório do GitHub indicado ao final do relatório. Os valores de teste foram escolhidos para que o resultado seja fácil de verificar.

4 Desenvolvimento

Os códigos abaixo também estão disponíveis, em arquivos separados, no repositório do GitHub: **https://github.com/GCWKGSWTGW/Daedalus-Ahmes-Ramses**

Exercício 1

> *1) Faça um conjunto de sub-rotinas para zerar os registradores A, B e X individualmente, e uma função para zerar todos de uma vez.*

No Ramses, as sub-rotinas são chamadas com a instrução JSR, que grava o endereço de retorno no primeiro byte da sub-rotina e desvia para a instrução seguinte. Por isso cada sub-rotina começa com um DB 0 reservado e termina com um salto indireto para esse byte, o que devolve a execução a quem chamou. No Ahmes não existem os registradores B e X, nem JSR, então o exercício foi adaptado com blocos rotulados que zeram o acumulador e duas variáveis de memória.

Código para o processador Ahmes

```
; Ex.1 - Ahmes: o unico registrador e o acumulador (AC)
org 128
VAR1: DB 5            ; variavel de teste (faz o papel de "B")
VAR2: DB 9            ; variavel de teste (faz o papel de "X")
ZERO: DB 0            ; constante zero

org 0
    LDA VAR1          ; AC = 5 (so para ver o AC zerar depois)
ZERA_AC:              ; bloco 1: zera o acumulador
    LDA ZERO          ; AC = 0
ZERA_V1:              ; bloco 2: zera a variavel VAR1
    LDA ZERO
    STA VAR1          ; VAR1 = 0
ZERA_V2:              ; bloco 3: zera a variavel VAR2
    LDA ZERO
    STA VAR2          ; VAR2 = 0
    HLT               ; fim (os tres blocos em sequencia zeram tudo)
```

O programa carrega VAR1 no acumulador apenas para que haja um valor diferente de zero. Em seguida, cada bloco carrega a constante ZERO: o primeiro zera o próprio acumulador; o segundo e o

terceiro gravam zero em VAR1 e VAR2 com STA. Executados em sequência, os três blocos equivalem à "função que zera tudo". Como o Ahmes não possui JSR, não há retorno automático, e a organização é feita pela ordem dos blocos.

Código para o processador Ramses

```
; Ex.1 - Ramses: registradores A, B e X e sub-rotinas com JSR
org 0
INICIO:
    LDR A #5          ; valores de teste diferentes de zero
    LDR B #6
    LDR X #7
    JSR ZERA_A        ; zera somente A
    JSR ZERA_B        ; zera somente B
    JSR ZERA_X        ; zera somente X
    LDR A #5          ; recarrega os tres registradores
    LDR B #6
    LDR X #7
    JSR ZERA_TUDO     ; zera A, B e X de uma vez
    HLT

ZERA_A:    DB 0       ; 1o byte: guarda o endereco de retorno (JSR)
    LDR A #0
    JMP ZERA_A,I      ; retorno (salto indireto)
ZERA_B:    DB 0
    LDR B #0
    JMP ZERA_B,I
ZERA_X:    DB 0
    LDR X #0
    JMP ZERA_X,I
ZERA_TUDO: DB 0
    LDR A #0
    LDR B #0
    LDR X #0
    JMP ZERA_TUDO,I
```

Os registradores A, B e X recebem valores de teste (5, 6 e 7). Cada chamada JSR leva a uma sub-rotina que carrega o valor imediato 0 no registrador correspondente e retorna com JMP rótulo,I. Depois da zeragem individual, os registradores são recarregados e ZERA_TUDO zera os três de uma só vez. Observa-se no simulador que o registrador alvo volta a 0 após cada chamada.

Resultado obtido: ao final de cada sub-rotina o registrador correspondente vale 0 (Ramses); VAR1 = 0 e VAR2 = 0 (Ahmes).

Exercício 2

> *2) Faça um programa para somar duas variáveis, em 8 bits e 16 bits, tratando o overflow de modo compatível ao tamanho da representação, armazenando em uma posição de memória (utilizar a declaração de variável) se ocorreu overflow (valor FFh) ou não (valor zero).*

Para números sem sinal, o overflow de uma soma ocorre quando ela gera carry (flag C). Na soma de 8 bits basta testar o carry após o ADD. Na soma de 16 bits, os operandos são representados por dois bytes (alto e baixo): somam-se primeiro os bytes baixos e, se houver carry, ele precisa ser incluído na soma dos bytes altos, pois nenhuma das duas máquinas possui "soma com carry". O carry gerado na soma dos bytes altos indica o overflow de 16 bits.

Código para o processador Ahmes

```
; Ex.2 - Ahmes: soma de 8 e de 16 bits com flag de overflow
org 128
```

```
N1:   DB 200          ; 1o operando (8 bits)
N2:   DB 100          ; 2o operando (8 bits)
S8:   DB 0            ; soma de 8 bits
OV8:  DB 0            ; flag de overflow 8 bits (FFh = estouro)
AH:   DB 2            ; 1o operando 16 bits - byte alto
AL:   DB 200          ; 1o operando 16 bits - byte baixo
BH:   DB 1            ; 2o operando 16 bits - byte alto
BL:   DB 100          ; 2o operando 16 bits - byte baixo
SH:   DB 0            ; soma 16 bits - byte alto
SL:   DB 0            ; soma 16 bits - byte baixo
OV16: DB 0            ; flag de overflow 16 bits
ZERO: DB 0
UM:   DB 1
FFH:  DB 255


org 0
    LDA N1
    ADD N2            ; soma 8 bits; o carry indica estouro
    STA S8
    JC OVER8          ; carry = 1 -> overflow
    LDA ZERO
    STA OV8           ; sem overflow: flag = 0
    JMP SOMA16
OVER8:
    LDA FFH
    STA OV8           ; com overflow: flag = FFh
SOMA16:
    LDA AL
    ADD BL            ; soma dos bytes baixos
    STA SL
    JC VAI_UM         ; houve "vai um" para o byte alto
    LDA AH
    ADD BH            ; soma dos bytes altos
    STA SH
    JC OVER16         ; carry final = estouro de 16 bits
    JMP OK16
VAI_UM:
    LDA AH
    ADD BH
    JC OVER16
    ADD UM            ; acrescenta o "vai um" do byte baixo
    STA SH
    JC OVER16
OK16:
    LDA ZERO
    STA OV16          ; sem overflow
    HLT
OVER16:
    LDA FFH
    STA OV16          ; com overflow
    HLT
```

Os valores 200 + 100 somam 300, que não cabe em 8 bits: o ADD ativa o carry, o programa desvia para OVER8 e grava FFh em OV8, enquanto S8 guarda 44. Para 16 bits (02C8h + 0164h), a soma dos bytes baixos gera carry (200 + 100 = 300), então o ramo VAI_UM soma os bytes altos e mais 1, resultando em SH = 4 e SL = 44 (042Ch = 1068). Como não há carry no byte alto, OV16 recebe 0.

Código para o processador Ramses

```
; Ex.2 - Ramses: soma de 8 e de 16 bits com flag de overflow
org 128
N1:   DB 200
N2:   DB 100
S8:   DB 0
OV8:  DB 0
AH:   DB 2
AL:   DB 200
BH:   DB 1
```

```
BL:   DB 100
SH:   DB 0
SL:   DB 0
OV16: DB 0

org 0
    LDR A N1
    ADD A N2          ; soma 8 bits
    STR A S8
    JC OVER8          ; carry -> overflow
    LDR A #0
    STR A OV8         ; sem overflow: 0
    JMP SOMA16
OVER8:
    LDR A #255
    STR A OV8         ; com overflow: FFh
SOMA16:
    LDR A AL
    ADD A BL          ; byte baixo
    STR A SL
    JC VAI_UM
    LDR A AH
    ADD A BH          ; byte alto
    STR A SH
    JC OVER16
    JMP OK16
VAI_UM:
    LDR A AH
    ADD A BH
    JC OVER16
    ADD A #1          ; soma o "vai um"
    STR A SH
    JC OVER16
OK16:
    LDR A #0
    STR A OV16
    HLT
OVER16:
    LDR A #255
    STR A OV16
    HLT
```

A lógica é idêntica à do Ahmes, trocando LDA/STA por LDR A/STR A e usando o modo imediato (#) para as constantes 0 e 255, o que dispensa variáveis auxiliares como ZERO, UM e FFH.

Resultado obtido: OV8 = FFh; S8 = 2Ch (44); SH = 04h; SL = 2Ch (44); OV16 = 00h.

Exercício 3

> *3) Limpeza de uma área de memória: faça um programa para zerar n posições consecutivas na memória, sendo a posição de início como uma variável declarada.*

O desafio é acessar posições de memória cujo endereço está em uma variável. No Ramses isso é resolvido com o endereçamento indexado: o registrador X recebe o endereço inicial e a instrução STR A 0,X grava na posição "0 + X". No Ahmes, que só tem endereçamento direto, o endereço é gravado dentro da própria instrução STA, que é escrita byte a byte com DB.

Código para o processador Ahmes

```
; Ex.3 - Ahmes: zerar N posicoes a partir de INI (codigo automodificavel)
org 128
INI:  DB 150          ; endereco inicial da area
N:    DB 5            ; quantidade de posicoes
ZERO: DB 0
UM:   DB 1
```

```
org 150               ; area de teste (valores nao nulos)
    DB 1
    DB 2
    DB 3
    DB 4
    DB 5
    DB 6
    DB 7
    DB 8

org 0
    LDA N
    JZ FIM            ; se N = 0, nada a fazer
    LDA INI
    STA PTR           ; grava o endereco inicial dentro do STA abaixo
LACO:
    LDA ZERO
    DB 16             ; opcode da instrucao STA
PTR: DB 0             ; operando do STA (endereco, alterado em execucao)
    LDA PTR
    ADD UM
    STA PTR           ; avanca para a proxima posicao
    LDA N
    SUB UM
    STA N             ; N = N - 1
    JNZ LACO          ; repete enquanto N diferente de zero
FIM:
    HLT
```

O byte DB 16 é o opcode de STA e o byte seguinte (PTR) é o seu operando, ou seja, o endereço onde o zero será gravado. O programa copia INI para PTR, grava zero, incrementa PTR para apontar à próxima posição, decrementa N e repete enquanto N for diferente de zero. Com a verificação inicial de N = 0, o programa também funciona quando não há nada a limpar.

Código para o processador Ramses

```
; Ex.3 - Ramses: zerar N posicoes a partir de INI (enderecamento indexado)
org 128
INI: DB 150           ; endereco inicial
N:   DB 5             ; quantidade

org 150               ; area de teste
    DB 1
    DB 2
    DB 3
    DB 4
    DB 5
    DB 6
    DB 7
    DB 8

org 0
    LDR X INI         ; X = endereco inicial
    LDR B N           ; B = contador
    JZ FIM            ; se N = 0, termina
    LDR A #0          ; A = 0 (valor a gravar)
LACO:
    STR A 0,X         ; grava 0 na posicao apontada por X
    ADD X #1          ; proxima posicao
    SUB B #1          ; decrementa o contador
    JZ FIM
    JMP LACO
FIM:
    HLT
```

X guarda o endereço atual e B funciona como contador. A cada volta, STR A 0,X grava zero, X avança uma posição e B é decrementado; ao chegar a zero, o laço termina. A área de teste a partir de 150 começa com valores de 1 a 8 para tornar visível a limpeza.

Resultado obtido: as posições 150 a 154 ficam zeradas, e 155, 156 e 157 permanecem com 6, 7 e 8.

Exercício 4

> *4) Faça um programa para comparar três variáveis declaradas no setor de variáveis.*

A comparação foi feita por subtração: calcula-se (variável − maior atual) e observa-se a flag N. Se o resultado for negativo, a variável é menor e o maior atual é mantido; caso contrário, a variável passa a ser o novo maior. A variável MAIOR começa com VA e é atualizada nas comparações com VB e VC. Esse método é válido para valores até 127, já que acima disso o bit mais significativo é interpretado como sinal.

Código para o processador Ahmes

```
; Ex.4 - Ahmes: maior entre tres variaveis
org 128
VA:    DB 3
VB:    DB 5
VC:    DB 14
MAIOR: DB 0           ; guarda o maior valor encontrado

org 0
    LDA VA
    STA MAIOR         ; assume VA como o maior
    LDA VB
    SUB MAIOR         ; VB - MAIOR
    JN TESTA_C        ; negativo: VB menor, mantem MAIOR
    LDA VB
    STA MAIOR         ; senao VB passa a ser o maior
TESTA_C:
    LDA VC
    SUB MAIOR         ; VC - MAIOR
    JN FIM            ; negativo: VC menor, termina
    LDA VC
    STA MAIOR         ; senao VC e o maior
FIM:
    HLT
```

Com VA = 3, VB = 5 e VC = 14: VB − 3 é positivo, então MAIOR = 5; depois VC − 5 é positivo, então MAIOR = 14. O programa termina em HLT com o maior valor armazenado em MAIOR.

Código para o processador Ramses

```
; Ex.4 - Ramses: maior entre tres variaveis
org 128
VA:    DB 3
VB:    DB 5
VC:    DB 14
MAIOR: DB 0

org 0
    LDR A VA
    STR A MAIOR       ; assume VA como o maior
    LDR A VB
    SUB A MAIOR       ; VB - MAIOR
    JN TESTA_C        ; negativo: mantem MAIOR
    LDR A VB
    STR A MAIOR       ; VB e o novo maior
```

```
TESTA_C:
    LDR A VC
    SUB A MAIOR       ; VC - MAIOR
    JN FIM
    LDR A VC
    STR A MAIOR       ; VC e o novo maior
FIM:
    HLT
```

Mesma lógica, usando o registrador A para a subtração (SUB A MAIOR) e o salto JN para decidir se o valor é menor.

Resultado obtido: MAIOR = 0Eh (14).

Exercício 5

> *5) Faça sub-rotinas para dado um número em byte (DB) e em palavras (DW) para transformar um número positivo, de um número negativo em representação de complemento de dois e em sinal-magnitude.*

O complemento de dois de um número é obtido invertendo todos os bits (NOT) e somando 1. Na representação sinal-magnitude, o bit mais significativo indica o sinal e os demais guardam o módulo; para obter o negativo basta ligar esse bit com OR 80h. Para a palavra de 16 bits, foram usados dois bytes (alto e baixo): o +1 é feito no byte baixo e, se houver carry (o que só ocorre quando o byte baixo era 0), é propagado ao byte alto. No sinal-magnitude, o bit de sinal fica no byte alto.

Código para o processador Ahmes

```
; Ex.5 - Ahmes: complemento de dois e sinal-magnitude (byte e palavra)
org 128
NB:   DB 3            ; numero positivo de 8 bits
C2B:  DB 0            ; resultado: complemento de dois (byte)
SMB:  DB 0            ; resultado: sinal-magnitude (byte)
NH:   DB 0            ; palavra de 16 bits - byte alto
NL:   DB 3            ; palavra de 16 bits - byte baixo
C2H:  DB 0            ; complemento de dois (palavra) - alto
C2L:  DB 0            ; complemento de dois (palavra) - baixo
SMH:  DB 0            ; sinal-magnitude (palavra) - alto
SML:  DB 0            ; sinal-magnitude (palavra) - baixo
MASC: DB 128          ; 10000000b: liga o bit de sinal
UM:   DB 1

org 0
    LDA NB            ; --- BYTE ---
    NOT               ; inverte os bits
    ADD UM            ; +1 = complemento de dois
    STA C2B
    LDA NB
    OR MASC           ; liga o bit de sinal
    STA SMB

    LDA NL            ; --- PALAVRA ---
    NOT
    ADD UM            ; complemento do byte baixo
    STA C2L
    JC VAI_UM         ; carry: byte baixo era 0, propaga o +1
    LDA NH
    NOT
    STA C2H           ; sem carry: so inverte o byte alto
    JMP SINAL
VAI_UM:
    LDA NH
    NOT
```

```
    ADD UM          ; byte alto recebe o +1
    STA C2H
SINAL:
    LDA NH
    OR MASC         ; bit de sinal no byte alto
    STA SMH
    LDA NL
    STA SML         ; byte baixo permanece igual
    HLT
```

Para o byte 3 (00000011b), NOT resulta em 11111100b e somando 1 obtém-se FDh (−3 em complemento de dois); com OR 80h obtém-se 83h (−3 em sinal-magnitude). Para a palavra 0003h, o byte baixo vale FDh sem gerar carry, o byte alto vira FFh e o resultado é FFFDh; em sinal-magnitude, o resultado é 8003h.

Código para o processador Ramses

```
; Ex.5 - Ramses: complemento de dois e sinal-magnitude (byte e palavra)
org 128
NB:   DB 3
C2B:  DB 0
SMB:  DB 0
NH:   DB 0
NL:   DB 3
C2H:  DB 0
C2L:  DB 0
SMH:  DB 0
SML:  DB 0
MASC: DB 128

org 0
    LDR A NB        ; --- BYTE ---
    NOT A           ; inverte os bits
    ADD A #1        ; +1 = complemento de dois
    STR A C2B
    LDR A NB
    OR A MASC       ; liga o bit de sinal
    STR A SMB

    LDR A NL        ; --- PALAVRA ---
    NOT A
    ADD A #1
    STR A C2L
    JC VAI_UM       ; carry: propaga o +1 para o byte alto
    LDR A NH
    NOT A
    STR A C2H
    JMP SINAL
VAI_UM:
    LDR A NH
    NOT A
    ADD A #1
    STR A C2H
SINAL:
    LDR A NH
    OR A MASC       ; bit de sinal no byte alto
    STR A SMH
    LDR A NL
    STR A SML
    HLT
```

A mesma lógica, usando o registrador A e a instrução NOT A. O modo imediato dispensa a constante UM.

Resultado obtido: C2B = FDh; SMB = 83h; C2H:C2L = FFh:FDh; SMH:SML = 80h:03h.

Exercício 6

> *6) Utilizando do código base ASCII, escreva seu nome completo a partir de uma posição de memória definida em uma variável declarada.*

O nome “GABRIEL CABRAL WALLENDZU” foi armazenado como tabela de códigos ASCII, terminada em zero, e um laço copia cada caractere para a região de destino cujo endereço inicial está na variável (POS no Ahmes e PTR no Ramses). A tabela ASCII associa a cada letra um número, por exemplo G = 71, A = 65 e o espaço = 32.

Código para o processador Ahmes

```
; Ex.6 - Ahmes: escreve o nome a partir da posicao POS
org 128
NOME:                ; tabela ASCII: GABRIEL CABRAL WALLENDZU
    DB 71    ; 'G'
    DB 65    ; 'A'
    DB 66    ; 'B'
    DB 82    ; 'R'
    DB 73    ; 'I'
    DB 69    ; 'E'
    DB 76    ; 'L'
    DB 32    ; ' '
    DB 67    ; 'C'
    DB 65    ; 'A'
    DB 66    ; 'B'
    DB 82    ; 'R'
    DB 65    ; 'A'
    DB 76    ; 'L'
    DB 32    ; ' '
    DB 87    ; 'W'
    DB 65    ; 'A'
    DB 76    ; 'L'
    DB 76    ; 'L'
    DB 69    ; 'E'
    DB 78    ; 'N'
    DB 68    ; 'D'
    DB 90    ; 'Z'
    DB 85    ; 'U'
    DB 0             ; terminador
POS: DB 200          ; posicao de destino
UM:  DB 1

org 0
    LDA POS
    STA DST          ; destino inicial dentro do STA
LACO:
    DB 32            ; opcode da instrucao LDA
SRC: DB 128          ; operando do LDA (endereco de NOME)
    JZ FIM           ; caractere 0 = fim do nome
    DB 16            ; opcode da instrucao STA
DST: DB 0            ; operando do STA (destino)
    LDA SRC
    ADD UM
    STA SRC          ; proximo caractere da origem
    LDA DST
    ADD UM
    STA DST          ; proxima posicao do destino
    JMP LACO
FIM:
    HLT
```

Como o Ahmes não possui endereçamento indexado, o laço usa instruções LDA (opcode 32) e STA (opcode 16) cujos operandos, SRC e DST, são incrementados a cada volta, caminhando pela tabela de origem e pelo destino. O laço termina ao ler o terminador 0 (a flag Z é ativada pelo próprio LDA).

Código para o processador Ramses

```
; Ex.6 - Ramses: escreve o nome a partir da posicao em PTR
org 128
NOME:                ; tabela ASCII: GABRIEL CABRAL WALLENDZU
    DB 71    ; 'G'
    DB 65    ; 'A'
    DB 66    ; 'B'
    DB 82    ; 'R'
    DB 73    ; 'I'
    DB 69    ; 'E'
    DB 76    ; 'L'
    DB 32    ; ' '
    DB 67    ; 'C'
    DB 65    ; 'A'
    DB 66    ; 'B'
    DB 82    ; 'R'
    DB 65    ; 'A'
    DB 76    ; 'L'
    DB 32    ; ' '
    DB 87    ; 'W'
    DB 65    ; 'A'
    DB 76    ; 'L'
    DB 76    ; 'L'
    DB 69    ; 'E'
    DB 78    ; 'N'
    DB 68    ; 'D'
    DB 90    ; 'Z'
    DB 85    ; 'U'
    DB 0             ; terminador
PTR: DB 200          ; posicao de destino (ponteiro)

org 0
    LDR X #0         ; X = indice na tabela
LACO:
    LDR A NOME,X     ; A = caractere atual
    JZ FIM           ; 0 = fim do nome
    STR A PTR,I      ; grava no endereco apontado por PTR
    LDR B PTR
    ADD B #1
    STR B PTR        ; avanca o ponteiro de destino
    ADD X #1         ; proximo caractere
    JMP LACO
FIM:
    HLT
```

No Ramses o código é mais direto: LDR A NOME,X lê o caractere indexado por X, e STR A PTR,I grava no endereço contido em PTR (endereçamento indireto). Depois, PTR e X são incrementados.

Resultado obtido: a partir da posição 200 ficam os bytes 71, 65, 66, 82, 73, 69, 76, 32, 67… que formam o nome.

Exercício 7

> *7) Proponha um código para multiplicação de dois números positivos.*

A multiplicação foi implementada por somas sucessivas: somar N1 consigo mesmo N2 vezes. Um contador é decrementado a cada soma e o laço termina quando ele chega a zero. O caso N2 = 0 é tratado no início, mantendo o produto em 0. O produto precisa caber em 8 bits (até 255).

Código para o processador Ahmes

```
; Ex.7 - Ahmes: multiplicacao por somas sucessivas (N1 x N2)
org 128
N1:  DB 4            ; multiplicando
```

```
N2:  DB 3            ; multiplicador (contador)
RES: DB 0            ; produto
UM:  DB 1

org 0
    LDA N2
    JZ FIM           ; se N2 = 0, o produto e 0
LACO:
    LDA RES
    ADD N1           ; RES = RES + N1
    STA RES
    LDA N2
    SUB UM
    STA N2           ; N2 = N2 - 1
    JNZ LACO         ; repete ate N2 chegar a 0
FIM:
    HLT
```

Com N1 = 4 e N2 = 3, o laço soma 4 três vezes: 4, 8 e 12. Ao final, RES = 12.

Código para o processador Ramses

```
; Ex.7 - Ramses: multiplicacao por somas sucessivas (N1 x N2)
org 128
N1:  DB 4
N2:  DB 3
RES: DB 0

org 0
    LDR B N2         ; B = contador
    JZ FIM           ; N2 = 0: produto 0
    LDR A #0         ; A = acumulador do produto
LACO:
    ADD A N1         ; A = A + N1
    SUB B #1         ; contador - 1
    JZ GRAVA         ; contador zerou: terminou
    JMP LACO
GRAVA:
    STR A RES        ; guarda o produto
FIM:
    HLT
```

No Ramses o contador fica no registrador B e o produto parcial no A. Como não existe JNZ, o programa usa JZ para sair do laço e JMP para repeti-lo.

Resultado obtido: RES = 0Ch (12).

5 Testes realizados

Todos os programas foram montados no Daedalus e executados nos simuladores Ahmes e Ramses, tanto no modo contínuo quanto passo a passo. Ao final de cada execução, foram conferidos o conteúdo da memória e dos registradores, e os valores obtidos coincidiram com os esperados em todos os exercícios, nas duas arquiteturas. A tabela resume as entradas utilizadas e as saídas verificadas.

Ex.	Entradas de teste	Saídas verificadas	Status
1	Ramses: A = 5, B = 6, X = 7. Ahmes: VAR1 = 5, VAR2 = 9	A, B e X = 0 após as sub-rotinas; VAR1 = VAR2 = 0	Aprovado

Ex.	Entradas de teste	Saídas verificadas	Status
2	8 bits: 200 + 100. 16 bits: 02C8h + 0164h	OV8 = FFh; S8 = 2Ch; SH:SL = 04h:2Ch; OV16 = 00h	Aprovado
3	INI = 150, N = 5; área 150–157 com os valores 1 a 8	Posições 150–154 = 0; 155–157 = 6, 7 e 8	Aprovado
4	VA = 3, VB = 5, VC = 14	MAIOR = 0Eh (14)	Aprovado
5	Byte = 3; palavra = 0003h	C2B = FDh; SMB = 83h; C2H:C2L = FFh:FDh; SMH:SML = 80h:03h	Aprovado
6	Posição de destino = 200	Posições 200 em diante: 71, 65, 66, 82, 73, 69, 76, 32, 67… (GABRIEL CABRAL WALLENDZU)	Aprovado
7	N1 = 4, N2 = 3	RES = 0Ch (12)	Aprovado

Em todos os casos o resultado vale para as duas máquinas (Ahmes e Ramses), com as diferenças de endereçamento descritas em cada exercício.

6 Divisão das tarefas

O trabalho foi realizado de forma individual. Todas as etapas — escrita dos códigos para as duas máquinas, montagem, testes, organização do repositório no GitHub e redação deste relatório — foram executadas por Gabriel Cabral Wallendzu (HT3080251). Para acompanhar o andamento, as tarefas foram organizadas em um quadro kanban, mostrado em seu estado final.

A fazer	Em andamento	Concluído
—	—	Ex. 1: zerar A, B e X (Ahmes e Ramses)
—	—	Ex. 2: soma 8/16 bits com overflow (Ahmes e Ramses)
—	—	Ex. 3: limpeza de memória (Ahmes e Ramses)
—	—	Ex. 4: maior de três variáveis (Ahmes e Ramses)
—	—	Ex. 5: complemento de dois e sinal-magnitude (Ahmes e Ramses)
—	—	Ex. 6: nome em ASCII (Ahmes e Ramses)
—	—	Ex. 7: multiplicação (Ahmes e Ramses)
—	—	Repositório no GitHub e relatório

7 Conclusão

A realização do laboratório permitiu observar, na prática, como a mesma tarefa pode exigir estratégias diferentes conforme a arquitetura. O Ramses, com mais registradores e modos de endereçamento (indexado, indireto e imediato), tornou mais simples tarefas como percorrer áreas de memória e chamar sub-rotinas. O Ahmes, limitado ao acumulador e ao endereçamento direto, exigiu soluções mais elaboradas, como a modificação do operando de uma instrução, o que evidencia a relação entre o programa e a organização da memória.

Também ficou claro o papel das flags: o carry no tratamento do overflow e na propagação do “vai um” entre bytes, a flag N nas comparações e a flag Z no controle dos laços. O uso do Daedalus tornou o ciclo de escrita, montagem e simulação mais prático e facilitou a verificação dos resultados.

8 Repositório e referências

Os códigos-fonte de todos os exercícios, para as duas máquinas, estão disponíveis no repositório do autor no GitHub:

https://github.com/GCWKGSWTGW/Daedalus-Ahmes-Ramses

Material de apoio da disciplina (Moodle do IFSP – Câmpus Hortolândia): apostilas dos processadores Ahmes e Ramses e do montador Daedalus.